

Making It Fast and Affordable

Where the money goes · Using fewer tokens · Cheaper models · Caching · Making it feel fast · When things fail

Where the money goes

You pay per token — roughly per word — for what goes in AND what comes out.

This costs you	Because
A long system prompt	It's sent with every single request
Whole documents pasted in	You pay for every word you send
Chat history	The whole conversation is resent each turn
Long answers	Output usually costs more than input



A prototype used by 5 people and a product used by 5,000 have very different bills. The fix is usually design, not negotiation.

Using fewer tokens

Do this	Why it works
Retrieve 3 pages, not 20	You're paying for every word you send
Summarise old chat turns	Stops long conversations growing forever
Say "answer in 3 bullets"	Output usually costs more than input
Trim the system prompt	You pay for it on every single call



Example

A support bot sending 15 documents per question instead of 3. Same answers — five times the bill.

Picking a cheaper model

The biggest model is rarely the right default.

Task	Model
Sort this ticket into a category	Small
Summarise this paragraph	Small
Explain this contract clause	Large
Write a customer apology	Large



Routing: send easy questions to the small model, hard ones to the large one. Most requests are easy — so the saving is real.

And if the smarts live in your search and your tools, a smaller model often does fine.

Caching

Lots of people ask the same things.

Type	What it saves
Prompt caching	Reuse a long instruction block you send every time
Semantic caching	Recognise a question you've already answered



"What's your returns policy?" asked 400 times a day.

Answer it once. Serve the other 399 from cache — instantly and free.



Cache the stable stuff. Never cache anything user-specific — "what's the status of my order" must not be cached.

Making it feel fast

What people notice isn't total time. It's the wait before anything happens.

-  Time to first token — how long until the first word appears
-  Streaming — show words as they're written, not all at the end
-  Faster search — a slow lookup delays everything after it
-  Shorter answers — finish sooner as well as cost less



A streamed answer that takes 8 seconds feels faster than a silent one that takes 4.

People will wait if they can see something happening. It's the blank screen that makes it feel broken.

When things fail

At scale, some requests get rejected. Plan for it.

Technique	What it does
Exponential backoff	Rejected? Wait, then retry — waiting longer each time
Connection pooling	Reuse connections instead of opening a new one each call
Cross-region inference	Busy region? Route the request to another one



Retrying immediately, over and over, makes it worse.

Everyone retries at once and the queue never clears. Backing off gives it room to recover.

Key takeaways

- 1 You pay for what goes in and out — the prompt is a running cost
- 2 Retrieve less · summarise history · ask for shorter answers
- 3 Route easy questions to a smaller model
- 4 Cache the stable answers, never the personal ones
- 5 Stream, so people can see something happening
- 6 Back off when you're throttled — don't hammer it

Working, fast and affordable are three different problems